

Lec 43 Practical Exercise - 1

Lec 43 Practical Exercise - 1

Lecture	43 / 63 + intro
Duration	50:54
Captions	English (auto-generated)
Words	7,845
Video	tZg9-lZnGBs
Playlist	Fundamentals of Generative AI and LLMs

Auto-generated YouTube captions, lightly cleaned (removed [music] tags, grouped into paragraphs). These are **verbatim lecture captions**, not edited notes.

Transcript

[00:00] Hello all, welcome to the hands-on series on fundamentals of generative AI and large language models theory and practice. In today's session, we'll be seeing the implementation of cycle GAN which is the cycle generative adversarial network which is a variant of the traditional GAN. In our previous sessions, we have seen the working of traditional GANs which makes use of one generator and a discriminator. From the image data set, what is given to a generator? It tries to generate fake images. Both the real image and the fake image when given to the discriminator, the discriminator tries to classify both of them and does some prediction. So the discriminator tries to classify whether the given image is a real image or a

[00:48] fake image. So in this process there is one generator and a discriminator and we when we talk about the laws pertaining to traditional GANs it is the generative laws as well as the discriminative laws. In case of cycle GAN which is a variant or advanced version of GANs, we make use of multiple generators and multiple discriminator because the entire process of cycle GAN is image to image translations. That is we try to have some images from domain A. We have images from domain B. Only domain A will be given during training to one of the generator. Domain B images will

be given to generator 2 while training. But the generator A should uh generate the images pertaining to domain B and generator B should image or should try

[01:41] to have the images or try to create fake images pertaining to domain A. So in this process what happens is the fake images created by generator A as well as B will be given to discriminator A and B. So totally the laws pertaining to the cycle GAN uh uh includes generator A loss, generator B loss, discriminator A, discriminator B loss and whatever process involved from A to B or B to A that is called as cycle consistency loss and how much identical images that are generated which is called as the identity loss. So all three losses that is the adversary loss, the cycle consistency loss and uh the identity loss. So put together will give us the loss loss pertaining to GANs. So this is one of the variant called as cycle GAN.

[02:37] Now let us see the implementation of cycle GAN. So the usual procedure is importing the necessary libraries. So uh let us make use of PyTorch which is one more uh open-source deep learning framework. Uh uh so far we have seen in our previous implementation we made use of Python programming language but we have not used the PyTorch. Instead we have used the TensorFlow or the Keras package. So let us have uh a different pytorch that is a deep learning framework in our implementation and let us see how is it different from the normal tensorflow or keras maybe in terms of the computational complexity or maybe in terms of the implementation or the execution time uh when it makes use of the CPU or the GPU. So let us start with

[03:26] the implementation. So importing the necessary libraries we import the pytorch which is a open-source deep learning uh framework generated or developed by meta or Facebook then we import uh from the pytorch or the torch we want the neural networks so when in short torch.nn is called as nn then we import uh optimizers from the pytorch framework that's why we have this optim as a short form for optimizers Then from the torch computer vision package we import the data sets as well as the transformations. So what data set we are trying to use here is the STL10 data set. STL 10 data set is a benchmark data set which can be used for image to image translation because it has 10 different classes of images of animals.

[04:18] So uh since we have to use only two domains here to have the cycle again, we are making use of horse images under domain A and deer images under domain B. So uh and STL 10 has more than 10,000 images. To reduce the computational complexity, we make use of only 400 images both in domain A as well as domain B. So then uh we import the data set which is that STL10 data set in order to have certain transformations on the image because we know that images uh do not are the raw images and it requires some kind of transformations maybe uh scaling of the images or rotation or some angle transformation or cropping of certain areas. That's why put together we have a package or we have a uh function called

[05:11] as transforms coming from PyTorch computer vision. Then from PyTorch

computer vision utilities we make use of grid. So grid is required in order to have uh multiple images in one particular figure. That's why we make use of grid. So that we may have 10 + 10 images in one particular grid or one particular uh figure. and then uh pytor utilities data we make use of the uh data loader [snorts] data set and the subset. So data loader is required if we are going for the deployment case.

[05:44] Suppose we are using some GPUs like RTX or Blackwell then we can use data loader but here let us see how uh we can use make use of this even though we have the T4 GPU and we make use of data set as well as subset. So why this data set is required because it's a readily available data set or from the UCR repository we are making use of this data set that's why we need this data set. Now coming to the subset as I already mentioned we wanted only two classes of data or two domains of data that's why we are extracting only a subset of the data that's why we make use of the subset function and for data visualization purpose we make use of matplotlib libplot in short we call it as plt and uh to generate some random

[06:30] number or random noise we need this random function as well as to uh ensure that this pytor works under uh the our required operating system we make use of OS as well. So let me make it clear here whenever we use PyTorch it is slightly different from our TensorFlow or Keras package and we should make sure whether it is compatible with our CPU or if uh not the GPU that that's why we make use of this operating system uh package as well as the data loader and make grid packages.

[07:06] uh also we are making sure if we have CPU connection or Z uh GPU that's why we have this uh additional two lines here device equal to from PyTorch we make use of device whether CPU is available or GPU is available so if GPU is not available then immediately it will switch on switch or convert into CPU so that it doesn't stop uh implementation so these are the initial uh importing of the libraries now coming to step number one that is uh initialization of certain values. So let the number of epochs be 20 in order to reduce the computational cost and let the learning rate of the model be 0.2. We have kept it very uh less because cycle GAN involves learning images which are unpaired unlike our

[07:55] traditional GANs. In case of traditional GANs we give the real image and we ask the generator to generate the fake images from the given domain. Unlike the case of cycle GAN, we give domain A images but the generator has to generate the fake images corresponding even to domain B. That's why we keep the learning rate as very slow otherwise it will converge very fast and in the process of converging to a very less loss value it may generate really fake images. uh that's why we keep it the keep it very low and the bat size is equal to four. So we consider only four images at a time. Now coming to step number two loading the data set. As mentioned earlier we are making use of STL 10 data set which is animal data set

[08:47] comprising of 10 classes. That's why the name 10 here out of which we

are choosing class number six which happens to be the horse data and also we are choosing class number four data which happens to be the deer data. So we configure or we treat horse images under domain A and deer images under domain B. Once we have uh loaded the data set, we have to do certain pre-processing on on the data set because the data set is a raw data set. So we try to compose all the transforms or transformations like resizing the image, cropping the image, converting the image into tensors or normalizing the image into uh one transform called as compost. So let us have uh one composition which comprises four different operations. So one after

[09:40] the other all these operations has to take place. So given image $256 + 256$ pixels we are resizing into $96 + 96$. So we are trying to compress the image first of all then cropping so that we have only 96 pixels in width and 96 pixels in height. So that is required soon after resizing and the main transformation what we are doing is converting into tensors because pytorch does not does not make use of numbers or does not make use of arrays instead it makes use of tensors. So when we talk about tensors the image representation which is uh the normal representation for a color image is width into height into channels or sometimes height into width into channels. But in case of tensors it is always the channel coming

[10:31] first followed by the height and then the width and then we try to normalize those values in the range minus1 to 1 because somewhere we can make use of the tanh function uh which uh where the values lie between minus1 to 1. That's why deliberately we try to normalize the values between minus1 to 1. So in the process of normalizing we do it for a color image that's why we have the value as three. The first value represents the mean that is 0.5 and the second value.5 represents the standard deviation. So normalized value will be x minus the mean value which is 0.5 divided by standard deviation which is 0.5. It is applicable to all three channels. So let us take one example and see how exactly

[11:18] this uh composition of transforms take place given any image. Let me take a $3 + 3$ image values lying between 0 to 255. So I may have value as 0 0 and then 10. Maybe this is 250 255 20 15 and 10. So this image which is which is ideally $256 + 256$ I have taken only $3 + 3$. This has to be resized into $96 + 96$. So once this is resized usually our normal representation is this happens to be width this happens to be height. Since it is RGB image we have three channels pertaining to red green and blue this representation will turn into three. So this represents the number of channels as well. So in case of pytorch the transformation will be channels followed by height and then the width. So what happens is this is 3 cross height will

[12:36] be 96 and the width will be 96. So this representation is the tensor representation. And now from this tensor representation if I uh take the tensor representation once again I have different values. Uh say this is 10 20 30 or 255 0 255 0 0 and 1. If I take this way then the normalization process will be this way. I'll take the first value. So normalized value for this 10 will be the actual value minus uh mean divided by

the standard deviation. So what happens for the value 10? It is value 10 minus mean which is 0.5 divided by 5 in our case. But this has to be repeated for the red channel, the green channel and the blue channel. That is why this has to be done into three. In the formula what we have seen in the code we have

[13:38] multiplied both the mean as well as the standard deviation with the value as three. So this is the initial pre-processing that we have to do on the STL 10 data set. So now uh printing of downloading the STL data set after all these transformations. So we can uh print that message download has to be true. If not uh throw the error message. Now coming to filtering of the horse and the deer image. So as mentioned earlier it has more than 400 image but we are comprising only up to 400. So 0 to 400.

[14:15] So it includes 0 to 399 which is totaling up to 400 images. So horse what is the horse data set or domain a data set? It is a subset of the all the data that we have downloaded for i for i and values having x as well as y in enumerate data if y = 6 is to 400. So we are taking subset of the data and every index varies from 0 to 399 in the class label six. In the process of taking all the data, we have to enumerate. That means we have to concatenate one value after the other.

[14:55] So this is the process of enumerating. Similarly, what is our domain B uh data? It is a deer data which is subset of the data. Once again, our index value varies from 0 to 39. We have to consider both the X and the Y values. That means we have to consider the figure as well as uh the label and we have to concatenate all the values or all the images. So likewise we have 400 image under domain A and 400 images in domain B. So we have to print how many uh are the domain A images and what is the length of the domain B images. So it will be 400 and 400 each.

[15:35] Now data set that randomly pairs one horse with one deer. So this is what I was mentioning uh earlier we have unpaired data because uh horse corresponds to y equal to 6. So class label is six for deer the class label is four. Now we have to create random pairs so that the generator generates the fake images. So how do we pair the data set? So we make use of domain A as well as B all the information or self information pertaining to domain A and B and we have to uh return the maximum value of the length of the data present in domain A as well as B and in every iteration we have all the images in A as well as B. So while pairing what we do is we do not consider the y part. We ignore the uh labels because the

[16:30] discriminator has to guess whether it is a deer image or a horse image. If it is a deer image whether it is a fake image or the real image. If it is a deer image whether the given image is real or fake. So that has to be guessed by the discriminator. So that's why the labels are ignored. So whenever we need to ignore anything in Python we make use of this underscore. So now self information or uh the information contained in domain a so it is based on the entire length of a and now randomly generate some integers starting from zero up to the length of uh all the images present in b

minus one. So `minus1` is required because the length will be 400. We want from 0 to 400 that is it becomes 4.1 images and

[17:20] then make it minus one so that so that we have values from 0 to index varies from 0 to 399. So finally return all the images of A and B both the domains. Now pairing the data set. So we pair both the horse and the deer data set and we uh consider it is in the loader called as data loader or simply a loader. And when we pair the data set we make use of batch size so that we need not go through all the 400 images in one stretch. Instead we can have batch size so that if we have a batch size as 64 first 64 images will be considered uh the epoch or the training of the data uh takes place on those 64 images then weights will be updated for those 64 images. So instead of having a very complex model we can uh restrict to batch

[18:10] size. batch size as 54 or 64 but in our case we have taken only four images at a time and `shuffle` equal to `true` it is very much required because uh otherwise there will be memory mapping that is image A or discriminator A knows that the image has come from domain A discriminator B knows that the image has come from generator A. So in order to avoid that memory mapping or memory copying we can say uh we have to make the images shuffle so that there is no one neuron understanding the memory units of the other neuron. So in that process we want the number of workers as two because batch size is four. Since batch size is four we want in multiples of two only.

[18:57] Now printing of batches per epoch. how many batches we are consider for every epoch. Now coming to uh having the generator and the discriminator. So as mentioned earlier in the cycle GAN we make use of two generators and two discriminator. So we can call this as suppose domain A has horse image. We have generator A which has to create fake deer images. Then we have discriminator which tells if it is a fake image or a real image. Now real image is domain A image fake image is B. So what how do we uh generalize this term is this is discriminator BA but this is generator AB because real images are A it has created fake deer images. So this called this can be called as generator AB and discriminator B ba. So when we do it

[20:15] vice versa what we do is we take domain B images which are the deer images. We consider generator B because it has to it is getting the information from domain B but it has to create fake images from domain A. So this is generator BA. So somewhere it generates fake horse image which the discriminator has to learn. So discriminator A. So totally we have four losses. Generator AB loss, discriminator BA loss, generator BA loss and discriminator AB loss. So these are the four laws and put together this can be called as cycle consistency loss because it's a cycle of domain A going up to domain B or we start with domain B and come to domain A. So this is a cycle of processes which involves the generator

[21:15] and the discriminator. So put together this kind of loss can be called as cycle

consistency loss. So now let us set up generator AB discriminator AB. So what is this generator? So we create a module called as generator from the neural networks. That's why we have a class generator. Uh and the module name is got from the uh neural networks. So it it comprises we always know a generator comprises of an encoder and a decoder. So initially we compress the image and then we decompress the image. nothing but a reconstruction takes place. So what does our comp uh encoder comprise of? Encoder will compress the image. So we initially start with the convolution process. We normalize those images and then one more

[22:08] convolution. So we start with uh the number of channels as three, number of neurons as 64, number of filters as seven and padding equal to three. Then we have the batch normalization and batch normalization value is equal to 64 and rail activation function because we have to avoid uh whichever negative values we have because always images should have the value from 0 to 255 only which are positive values. Uh relu is one activation function which makes use of maximum value of 0 and the actual pixel value. So if the actual pixel value is a negative value maximum of 0 and that negative value will be zero. If the pixel value is a positive value then 0 comma that positive pixel value will be a positive value. That's why we make

[22:57] use of reo activation function. Then the next convolution layer comprising of 64 neurons uh 128 uh filters such filters we use and then uh the filter size is three and we make use of stride equal to two. So we try to increase the stride so that we avoid uh one window in between the two strides and padding is necessary so that we don't uh lose the edge information. Now coming to one more normalization we increase the uh uh normalization size that is 128 neurons.

[23:31] Once again the rail activation function convolution 128 filters and then uh 2 128 neurons 256 filters 3 cross three is the window size stride being two padding equal to one. So we should always remember that the stride value or the padding value is left from one user to the other user because sometimes we uh try to overlap and uh fix onto stride equal to zero. Sometimes we need to avoid the uh in between content content between two different pixels or three different pixels. That's why we increase the stride value. So it is always the trial and error basis. So there is no logic be behind why to use a stride equal to two or stride equal to one. So whenever we are confident enough on uh the generator

[24:20] or we are confident enough on the data set what kind of data set we have we can always have a bigger or a larger stride. So finally we have this normalization making use of 256 neurons and the radio activation function. Now uh finally we have this transformer uh module. So convolution 256 neurons 256 window size and three channels we make use of and padding equal to one followed by the normalization and relu. So this is the transformer unit or the we can call it as a whatever transformation we have done on the original image. So likewise we do it four times because we have made use of three convolution. So we make use of four convolution here 3 + 1 always

whatever encoder we have created that is

[25:10] three convolution we what we have created for those three convolution we should now have the decoding part. Now what is the decoding part it is always the decompression. So in case of compression we start with a small number small number of neurons and uh reach up to large neurons. So if we see here three neurons 64 neurons and then 128 neurons or three filters 64 filters or we can call it as 128 filters. Now it should be the reverse way in case of decoder. We start with 256 then we come down to 128 and 64. So the entire process of encoding and decoding is such a way that the filtering is low-level filtering. when we start uh just start with the image then low-level filtering is done we go with the mid-level

[25:59] filtering and then the highle filtering so it is a deep uh feature extraction what happens in case of encoder now in case of decoder it has to be a reverse process that means whatever feature in uh extracted from that we have to uh gauge the patterns we have to find the patterns out of it so it becomes like a decompression or a deencocoding that's why we call it as a denoder or decoder in short called as decoder. So finally we want all the values to lie between minus1 and 1. That's why we make use of the tanh activation function. So relu will have all the values relu activation function is required in order to avoid the negative values. So it functions in this way. It is maximum of 0 comma x. Suppose we have a negative

[26:57] value say minus2 then what happens is maximum of 0 comma minus2 it happens to be zero. See if we have a value as 10 a positive value maximum of 0a 10 it happens to be 10. So in both the cases the positive value is retained. So negative value converted into zero positive value retained as it is. So this is the radio activation function. Whereas tanh activation function always tries to have the value between minus1 and 1.

[27:28] So zero can be called as the mid or the median value. That's why we expect mean to be in and around zero and standard deviation somewhere around one so that it follows a gshian distribution something like this. So mean should be somewhere centered around zero and this could be plus or minus one standard deviation, two standard deviation or three standard deviation based on how much we are able to accept the outliers.

[28:01] So outliers means uh when we say outliers in terms of numbers it is corresponding to the fake or very fake images. So this is all about the encoder part. So this is all about the generator part. Now coming to the discriminator. Discriminator has to guess whether the given image is a real image or a fake image and it also has to guess whether it is from domain A or B. So how do we define the discriminator? Once again from the neural network module called discriminator we create a class called as discriminator. It is uh a layer by layer stacking of layers by layer.

[28:46] That's why we have the function called as sequential from the neural net-

works. So what is happening? We make use of four convolution layers. Once again this is uh low-level filtering to high level filtering. So in case of low-level filtering some of the features that could be extracted or guessed uh could be lines, points or edges. Mid-level filtering tries to have some patterns or maybe it can also focus on shape of the image or texture of an image. But while coming to high level filtering like 128 and uh 256, it is able to get uh some contours on the image or um the unique patterns. So suppose we talk about a zebra then there are unique uh contours called as contours of blue as well as black. So it is able to uh identify

[29:34] those contours or circles. Suppose we are talking about um about maybe cat it has fur. So based on the texture it is able to discriminate. So we start with a small point on an image and try to come up to the texture or the various contours present in the images. So that's why we have different levels of filtering lowle to high level filtering. Once again uh the stride and padding is left to the user. Uh we can use different values of striding how much we have to take the window size and we make use of the leaky relu function. So the difference between leaky relu and relu is relu will try to convert even the negative values into positive values.

[30:20] But in the process of converting uh some of the activation functions or some of the neurons may be lost or what we say is most of the values will become zero. So there is no learning which is happening. In case of leaky relu some leaky values or even a small alpha values will be tried to retain. Maybe the value could be 0001 uh due to which the uh the neuron will be active. So the activation function will be active nothing but the neuron will be active and there will be some processing happening on every neuron or every hidden unit which is present. That's why we make use of the leaky relu function.

[31:07] So now coming to creating the four models generator A generator BA discriminator A discriminator B. So this is already mentioned earlier. What is generator AB? What is BA? Similarly, what is discriminator A and discriminator B. So it is also uh given in terms of comments here. Generator AB means we have domain A horse images from which deer images will be will be generated that is the fake deer images. Generator BA means it takes the input from domain B which is deer and tries to create fake horse images. Discriminator A will ask or it will fetch for whether it is a real horse, whether the data is coming from domain A. What will discriminator B do is it is looking for real D that is images from domain B.

[32:00] So uh we have to send all the discriminator or the generator what we have considered to the CUDA device. That's why we have this function to device. So once this is done we have to print uh the number of parameters corresponding to generator as well as parameters corresponding to the discriminator. So we know that always uh there should be there will be some operations in different hidden layers uh except the input layer. Input layer takes only the input but after the input layer we have the hidden set of hidden layers multiple hidden layers and then we have the output or the

fully connected or the dense layers. So at those layers different operations will take place. So the weight updation is

[32:40] based on what inputs we have, how many hidden neurons we have plus whatever bias associated with those uh with those neurons. So based on that we have this parameter calculation. So let me show one simple example or the how do we mathematically represent those parameter calculation. So always we should remember uh the number of parameters will be equal to input whatever input that is going to that layer maybe input layer or hidden layer into the hidden neurons plus the bias associated with those neurons. So this is the parameter calculation. So such parameter calculation holds good both for the generator as well as the discriminator.

[33:30] So now uh coming to the loss functions there could be mean square error uh there could be L1 loss also. So mean square error is the GAN loss which is already mentioned uh which which can be called as a adversarial loss also the L1 loss which is the cycle plus the identity loss. So as mentioned earlier in the in the session there is cycle consistency loss as well as identity loss. So both can be called as L1 loss. Now what are the different optimizers related to the generator and the discriminator? So we have the one of the optimizer related to one of the optimizer related to the generator which is the Adam optimizer. So we have to list all the parameters that is calculated in the previous line here

[34:19] as well as all the parameters related to the second generator that is BA. and whatever learning rate that we have set that is 0.00001 and whatever beta that is the bias uh values that is set. Similarly the beta 1 and the beta 2 values we know that uh the adam optimizer is based on the mean calculation as well as the variance calculation. So in the process of mean calculation and variance calculation there are two scalar values or constant values associated with this that is one and the second one beta 1 and beta_2.

[34:56] Now coming to optimizer related to discriminator A. So all the parameters that are involved in the discriminator A plus its uh learning rate and beta 1 as well as beta 2 values. Similarly, even for discriminator B, we have beta 1 and beta_2 values. So, in the session of autoencoders, we have already seen why we should have uh beta value beta 1 value as 0.5 and why beta 2 value should be nearly equal to one somewhere like 10^{-9} or $10^{-9.98}$.

[35:31] So, we have already seen in case of autoencoders. » [clears throat] » Now uh we consider all the samples which are existing in our OS that is we have created a directory uh from uh the existing OS and we consider all the samples in order to uh optimize with the help of Adam optimizer. Now coming to the training part training on the training uh training on the samples that is the input samples. To print the starting of the training we can print the statement for every epoch in the range one and at every iteration it will be epoch + one. So we have to load all the horse images the

deer images from the loader and the real and the fake we have to have two different folders in order to comprise the real image and the fake

[36:28] image. So based on the real and the fake image then we have the loss calculation. So real image we have to append ones. Fake image we have to append zeros. That means in case of real images and fake images we have to start concatenating one. So if until we have the real images the dense vector will have values as one only. Suppose one fake image comes in between then the dense vector value will be zero. Same thing for the fake also we should create one dense vector. So if it is real then definitely this value will be one. So these many values will be one. Since this is one the opposite will be zero. So this is a vector corresponding to the real image and this one to the fake image.

[37:22] So now coming to training the generator. So as mentioned earlier there are two different generators. one for the horse, other one for the deer. Based on whether image is coming from domain A or B, we have four combination. So horse to fake deer generation, deer to fake horse generation. Similarly, fake deer back to horse generation, fake horse that is created back to deer generation. So we have four different uh combinations. So this will whatever loss associated with respect to reconstruction it is the reconstruction loss for the horse and reconstruction loss for the deer.

[38:03] So put together the losses are the mean square error loss corresponding both the two discriminator B and discriminator A and the cycle uh uh consistency loss that is that can be called as L1 loss corresponding to the reconstruction loss as well as the real image. Likewise we have 10 different classes. Similarly, L1 of that is reconstruction loss for the deer as well as the real deer images comprising of 10 different classes. And the identity loss will be whatever generator we consider that is the AB generator and the uh deer image that is generated plus the actual deer images because how much identical it is.

[38:45] Similarly, L1 corresponding to horse image that is generated and how many real horse images were there. So this is this shows the identity or similarity that's why the name is identity loss. So total gan loss for a cycle gan loss will be total gan loss that is a mean square error loss plus the uh consistency loss plus the identity loss. So in the backward propagation we have to take this loss of the GAN and then the optimizer whatever optimizer in our case the adam optimizer that has been used it will update the weights. So the relationship between uh the optimizer and the gradient values or the learning rate is this way.

[39:32] we have the new weights that is calculated in the back propagation that will be corresponding to the old weight minus η into $\frac{\partial L}{\partial W}$. So this $\frac{\partial L}{\partial W}$ corresponds to the gradients. So how much loss incurring due to the change in the weights that has to be calculated. So that is back propagated. So how much change

in the weights has to be done. What is a new weight? So that depends on the old weight minus the EA value and DL by DW depends on optimizer and our optimizer is Adam optimizer which is based on beta 1 and beta 2 values. So totally what is uh training of discriminator A. What is the training involved in discriminator B? So training of discriminator A means it is the mean square uh error associated with

[40:26] discriminator A which takes the real image into account. mean square error for discriminator A which takes the uh fake horse image into account. So multiplied by 0.5 because we also have the discriminator B. So two two different discriminator. Similarly discriminator B means uh one which is associated with the domain B. We have the mean square error for the deer images that is coming from uh domain B plus the real images which are already generated by the generator.

[40:59] Similarly, mean square error for the fake deer images that is uh created and the fake images. Since we are considering both the discriminator, the weightage is given equally 0.5 and 0.5. So finally printing of the epoch finding of uh printing of the generator loss discriminator loss. So put together we have one generator loss now which considers the adversarial loss, cycle consistency loss and the identity loss. So showing the results after every epoch.

[41:29] So this is required while printing of the image and the epoch values. So we uh consider the different iterations which is based on how many images we are loading and every iteration it has to pass on to the next image. So that's why we have this function called as next. So we have four images to be displayed in every row. That's why we have index is to four. So 0 to 3. So 0 1 2 3 which corresponds to the real image coming from one of the domain. Fake images created by the generator that takes the second position. Uh whatever image uh the discriminator gets that is takes the third uh placeholder and the fourth image will be whether the discriminator is able to classify them. So that is the fourth placeholder. That's why we have

[42:20] four placeholders here. So finally we have forward and backward propagation. In the forward we have the deer image, the horse image and in the reconstruction the horse image and the deer image. So we have we can see it in the comments horse to deer, deer to horse, cycle back to horse and cycle back to deer. So this process is already explained uh in the uh text editor. Now coming to displaying what are what are the different uh uh images that we need to see in the while we plot the images. So we have the real horse, fake deer and reconstructed horse. Similarly, original image is always there. Then we have the real deer, the fake horse that is generated and the reconstructed deer.

[43:10] So we have to make a grid of these many things. So we have different categories from PyTorch. CAT stands for categories. So different categories are horse, fake deer, uh really con or reconstructed horse, deer, fake horse and reconstructed deer. So four images normalized value true because we have already normalized it in

the value as Z minus 1 to 1. Now to get back we have to have the tensor values. From the tensor values we have to get the pixel values.

[43:45] That's why we have to consider the normalized value. So let us have a figure of argument figure size uh with the width as 12 in and height as 6 in. So let us show in the form of a grid and uh permute. Permute means we want one row and two columns and different placeholders. So let the axis be off. Let us not give any naming for the x-axis or the y-axis. We have to just display the original fake image and the real image. That's why we keep the axis as off. and the different titles. Row number one, the original image, real, fake, and reconstructed. Similarly, row, row number two. Likewise, we want the title to be given. Uh, let the font size for the title be 10. So, all these are some of the arrangements or uh settings that

[44:35] we do for plotting. uh plot uh tight layout means uh it shouldn't go whatever images that we display shouldn't go out of the layout out of the layout of that is already described here we have already described as 12A 6 so we want to save the figure we can save it in our system that's why uh this kind of setting we are doing as a PNG with uh resolution as 100 then we have to show the image age. So training both the generator AB and BA. So all the printing to be done as and when we uh save the models that is generator and the discriminator after every epoch we should have a message as saving the models. So we are saving the dictionary values of generator AB the dictionary values of BA. So what is the dictionary

[45:33] consist of? It has a key and a value. Key means it shows which domain it has come from. Value means whether it is real or fake. So we create a dictionary of two items. One is the key and other one is the value. And uh whatever images generated by generator AB that is saved in this path that is G horsetoer translation. Whatever fake images that is generated by uh generator BA is saved in a path and the path name is dear to horse. So this is one more image translation. So basically we started with domain A images, domain B images separately. Now with the help of cycle GAN we have done a image translation A to B and B to A translation. So finally we have uh to print a message called as saved. We have saved both the paths.

[46:30] So whatever printing we have done uh we have the output for all of them. Uh downloading the STL data set STL 10 data set. We have horse uh images which are 400 and four deer images which are 400 in number. Number of batches we have taken per epoch is uh 100. Why? Because we have taken batch size as four and we have 400 images. So 400 divided by 4 we get it as 100. Number of generator parameters are 3.1 million parameters that we need to calculate and for the discriminator the number of parameters are 7 million. So once again uh let us recall parameter calculation is based on number of inputs that we are considering into the number of neurons associated with the uh hidden layers plus the bias associated with the

[47:19] hidden layers and uh we start the training. So we have to run 20 epochs

and these are the losses pertaining to generator discriminator A discriminator B. So what is happening at epoch one? We have the original image, the real horse, the fake uh fake deer image and the reconstructed horse. Similarly, the real deer, the original image, the real deer, the fake horse and the reconstructed deer. So likewise this display we have to continue up to the last epoch. So the different epochs here we can see the different epochs and the associated generator loss, discriminator A loss and discriminator B loss. So we have run 20 different epochs. We have made uh the model the cycle GAN model to run or go through the 400 images 20

[48:19] different times. So finally the generator loss is 5.12 the discriminator A loss is 2 and discriminator B loss is 16. So we say that the discriminator B is better when compared to A. So it is able to uh correctly classify the deer and the horse image. So we have saved the model for both the path as uh that is horse to deer as well as deer to horse. So this is required uh the GAN images or the images created by both the generators are required when we process onto new uh set of images. So we can use this as a base image and we'll be able to make use of other GANs and create some more fake images. That's why we should always save the fake images created by the generator. So this is all about the implementation of cycle GAN.

[49:14] cycle GAN which is cycle generative adversarial network. So as mentioned earlier cycle generator generative adversarial network is a variant of traditional GANs which makes use of multiple generators and multiple discriminator. So this is the way it is different from traditional GAN. So traditional GANs the reconstruction loss will be very high because we make use of only one generator and a discriminator unlike in case of a cycle GAN we try to uh train the real or the fake images by having multiple combinations of generator and discriminator. So when we try to compare this with the traditional GANs or the uh DC GANs what we have seen in our uh previous sessions this GAN is helpful in image to image translation.

[50:01] So when we say image to image translation it is uh required to have images only from one of the domain and with the knowledge that is uh acquired by the GAN from one of the domain it is able to have a fake image from the other domain. So in our case of STL 10 data set we have made use of only two domains horse domain and the deer domain. By taking only the horse image it is able to construct a deer image. By taking the images only of the deer domain, it is able to reconstruct or construct images pertaining to a horse. So this is called as unpaired data or uh cycle GAN makes use of unpaired data. So this is all about cycle GAN. Thank you.